

MoE

Mixture-of-Experts

东川路第一可爱猫猫虫

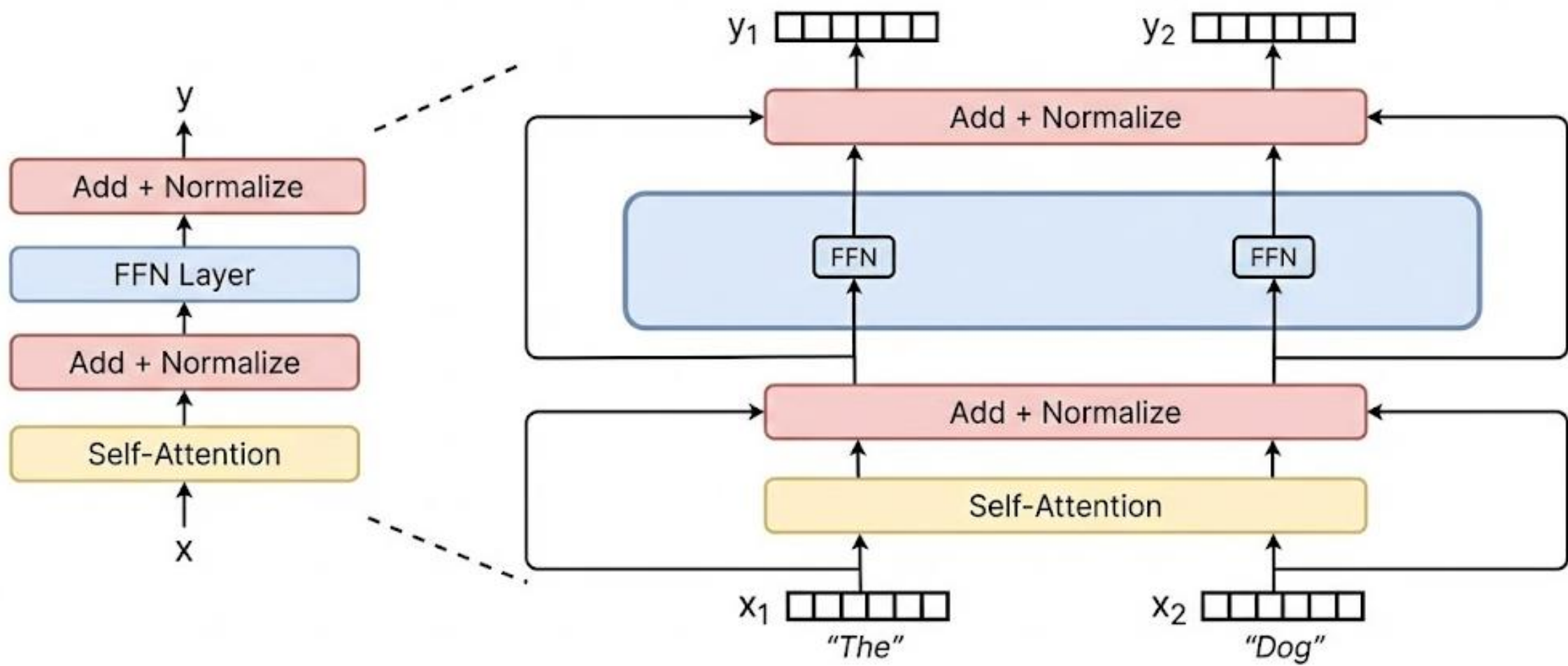
主要内容

- Switch Transformer
 - DeepSeek V1/V2 & Qwen & GROK
 - Auxiliary Load Balancing Loss
- DeepSeekMoE
 - 细粒度专家和共享专家
- DeepSeek V2
 - Device-Limited Routing
- DeepSeek V3
 - Auxiliary-Loss-Free Load Balancing

感谢粉丝大佬们这段时间的充电

- 狂且X
- 迷途小白哟
- 你的初心
- W0ND3RFULHE4VEN
- 宽人克己是謂上策
- 浪浪洗脚
- 阿萨李逍遥张小凡
- 蕾西亚万岁啊我透
- 你的初心
- Lib_Yancy
- 我们的征途是漫漫星海

标准transformer



Dense Model

- 每一个token都要进入同一个FFN层

每一次前向传播时会用FFN层所有参数计算

这里的计算量一般用FLOPs来衡量

当模型参数越多，权重矩阵越大

计算量就会越大

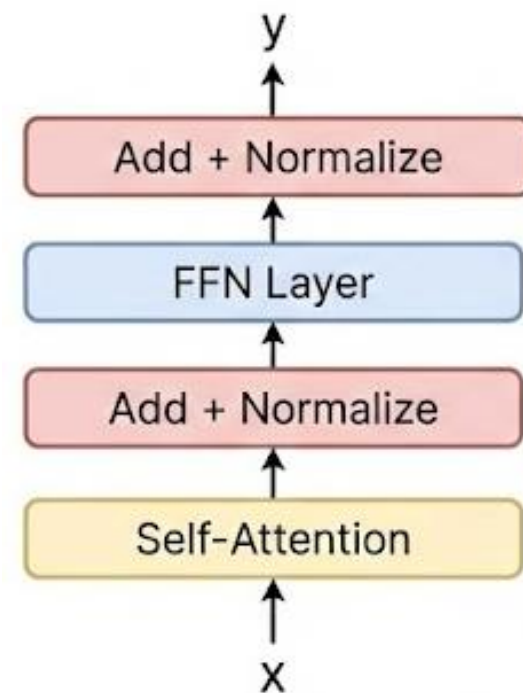
- MoE

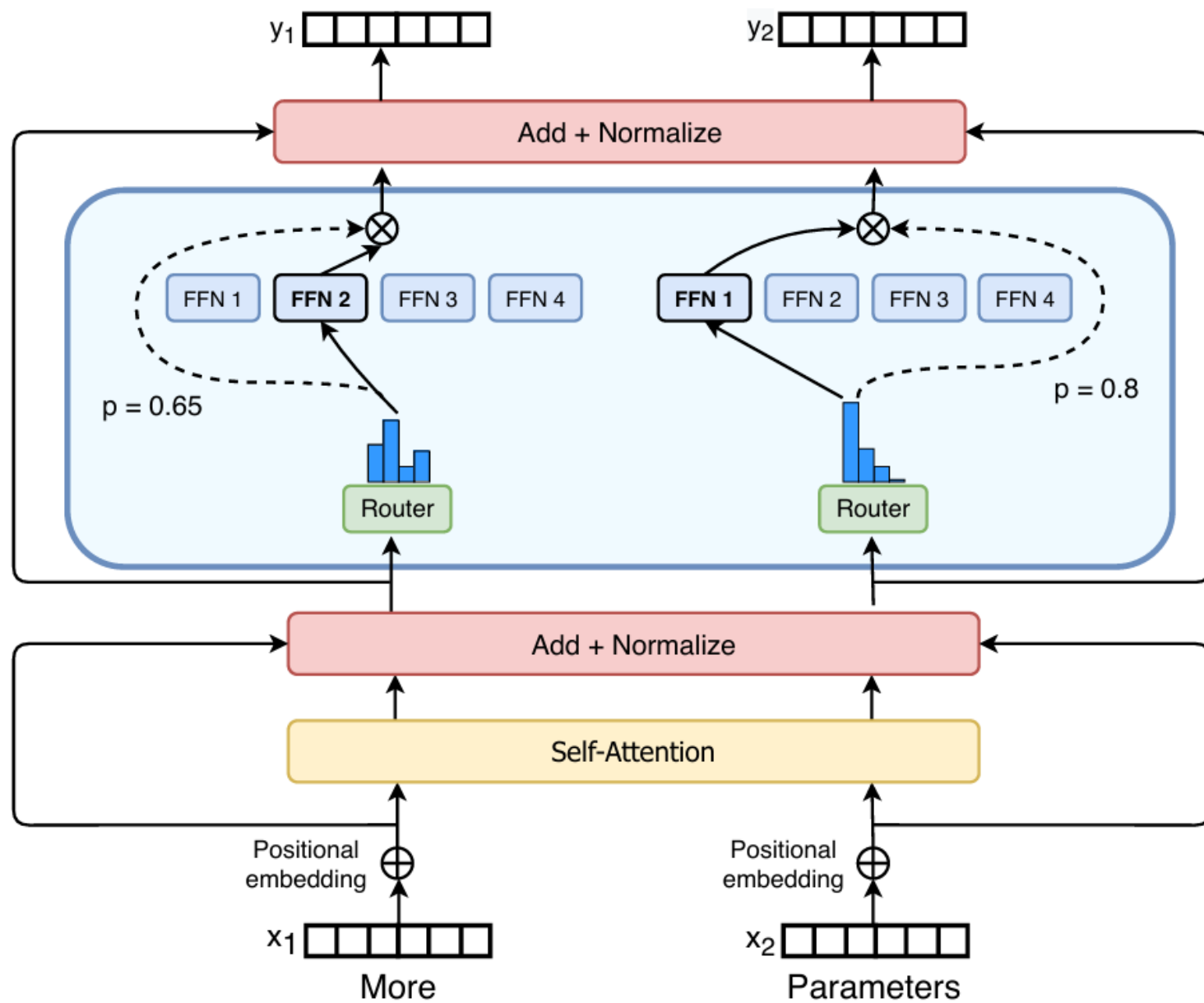
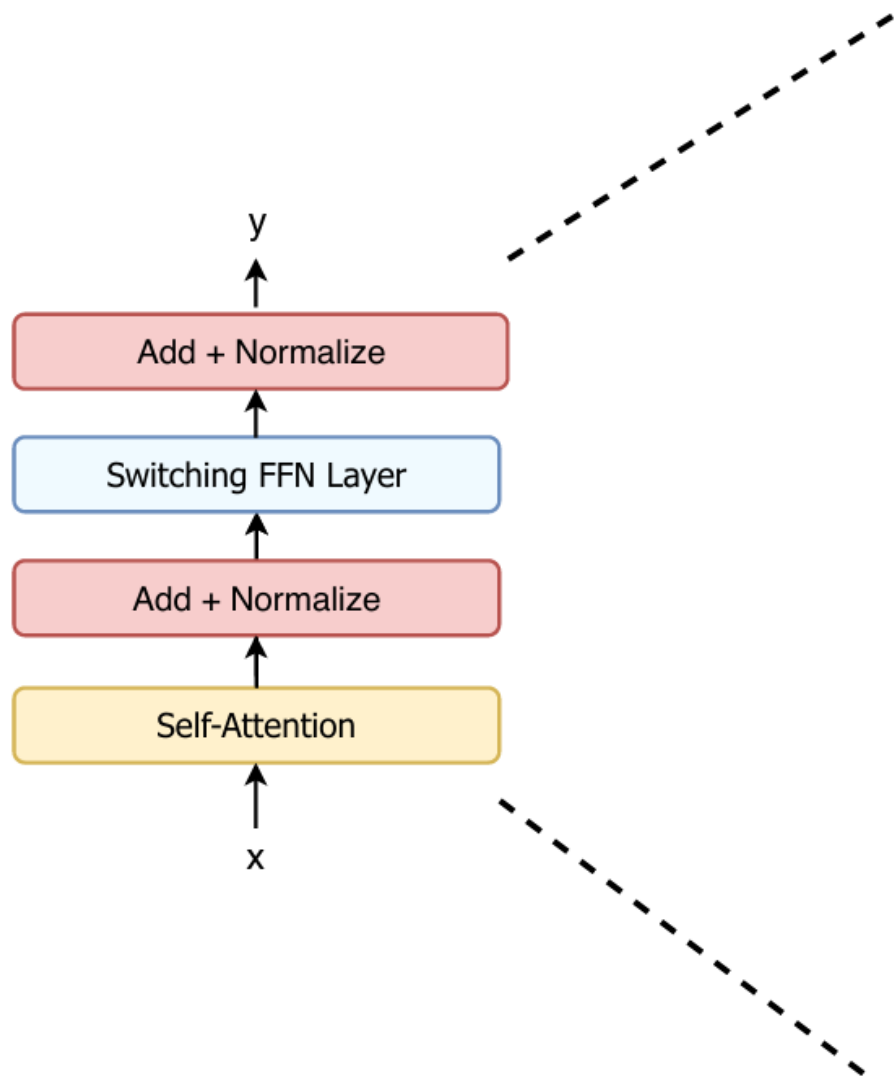
Mixture-of-Experts

提供了一种办法，在保持FLOPs几乎不变的情况下增加模型参数量

MoE把大的FFN切分或复制成多个FFN

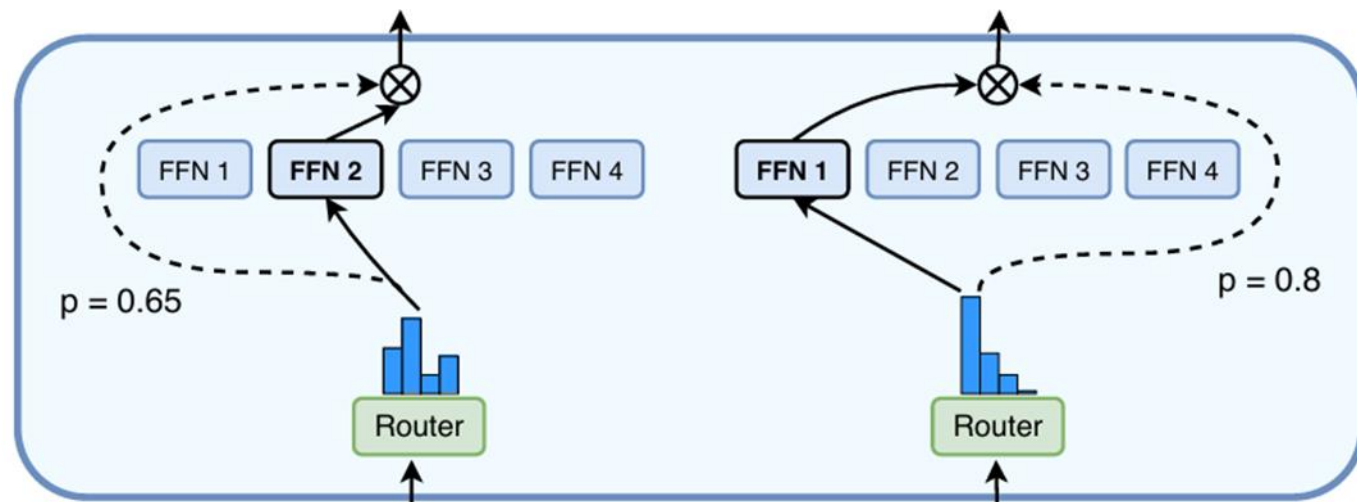
处理每个token时，只稀疏地激活其中的一个或几个





MoE 混合专家

- 每一个FFN就是一个专家
- 对于每一个token
 - 通过router选择最适合他的FFN
- token对专家的选择
 - 计算向量内积
 - token的向量与每个专家的向量相乘
 - 得到该token与各个专家的匹配度
- 专家最多学习到token级别的语法



router 路由器

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	1.32	1.97	0.1
	E4	2.61	2.61	0.02
	E5	2.81	0.68	0.41

Token chooses expert

		Tokens		
		T1	T2	T3
Experts	E1	Choose Top-K		
	E2	0.51	-0.25	1.58
	E3	1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	2.81	-0.68	-0.41

Expert chooses token

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.51	-0.25	1.58
	E3	1.32	1.97	0.1
	E4	2.25	2.61	0.02
	E5	2.81	-0.68	-0.41

Global routing via optimization

Token Choose Expert

- Switch Transformer
DeepSeek V1/V2 & Qwen & GROK
- DeepSeekMoE
细粒度专家和共享专家
- DeepSeek V2
- DeepSeek V3
保持这种Token Choose Expert
softmax after TopK

		Tokens		
		T1	T2	T3
Experts	E1	3.13	0.14	0.74
	E2	0.11	-0.25	1.58
	E3	0.32	1.97	0.1
	E4	0.5	2.61	0.02
	E5	2.81	0.68	0.41

Token chooses expert

Switch Transformers (Fedus 2022)

- $s_{i,t} = \text{Softmax}_i \left(\mathbf{u}_t^{\text{l}} \mathbf{e}_i^{\text{l}} \right)$

其中 $\mathbf{u}_t^{\text{l}}$ 表示第t个token的隐藏状态的向量

$\mathbf{e}_i^{\text{l}}$ 表示第i个专家的向量

我们让向量点积通过softmax归一化

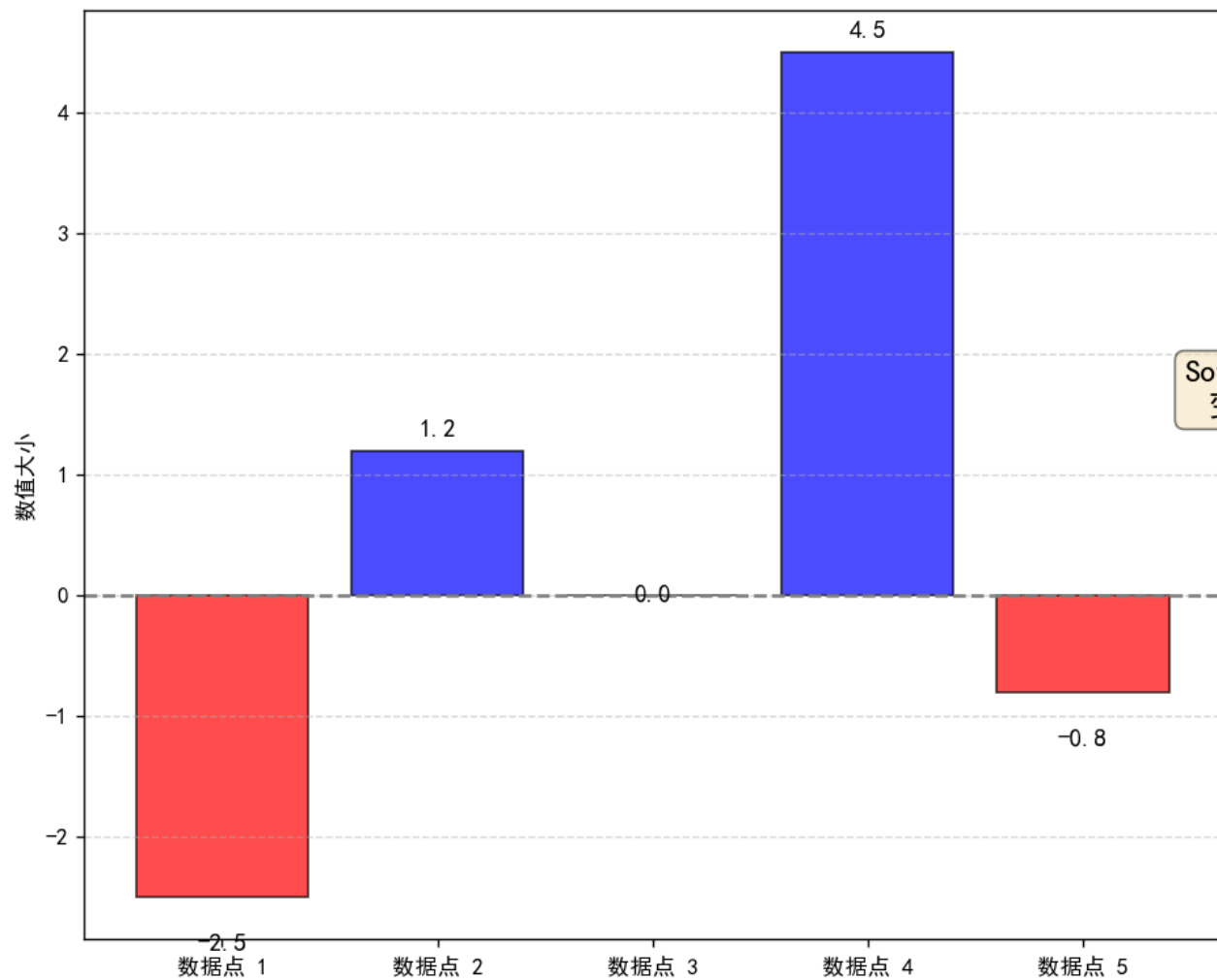
得到了token选择每个专家的概率

- $s_{i,t}$ 表示第t个token选择第i个专家的概率

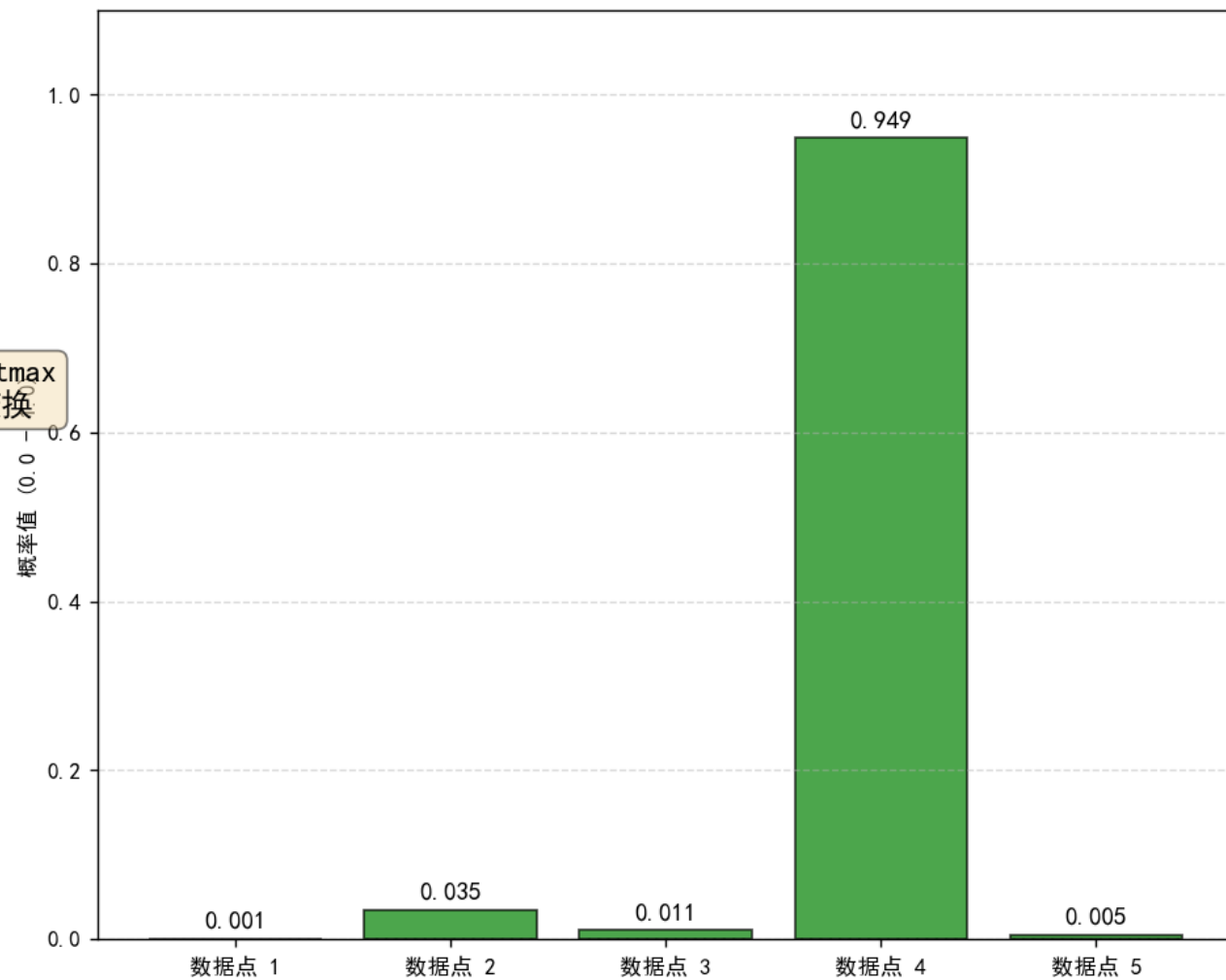
softmax是对于每个token来讲的

对于一个token，它和所有专家的匹配度之和为1

变换前：原始输入 (Logits)
(包含正负值，范围任意)



变换后：Softmax 输出 (概率)
(全部在 0-1 之间，总和为 1)



Softmax
变换

KeepTopK

- 对于每个token, 我们筛选出得分最高的K个专家
- 如果这个专家的得分属于top-K
就保留
否则置为0

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K) \\ 0, & \text{otherwise} \end{cases}$$

important for models with many routing layers. Contrary to these ideas, we instead use a simplified strategy where we route to only a *single* expert. We show this simplification preserves model quality, reduces routing computation and performs better. This $k = 1$ routing strategy is later referred to as a Switch layer. Note that for both MoE and Switch

$$\mathbf{h}_t^l = \sum_{i=1}^N \left(g_{i,t} \cdot \text{FFN}_i(\mathbf{u}_t^l) \right) + \mathbf{u}_t^l$$

- 其中 $g_{i,t}$ 是权重

每个选出的专家的输出根据权重加权求和
加上末尾的 $\mathbf{u}_t^l$ 完成残差连接

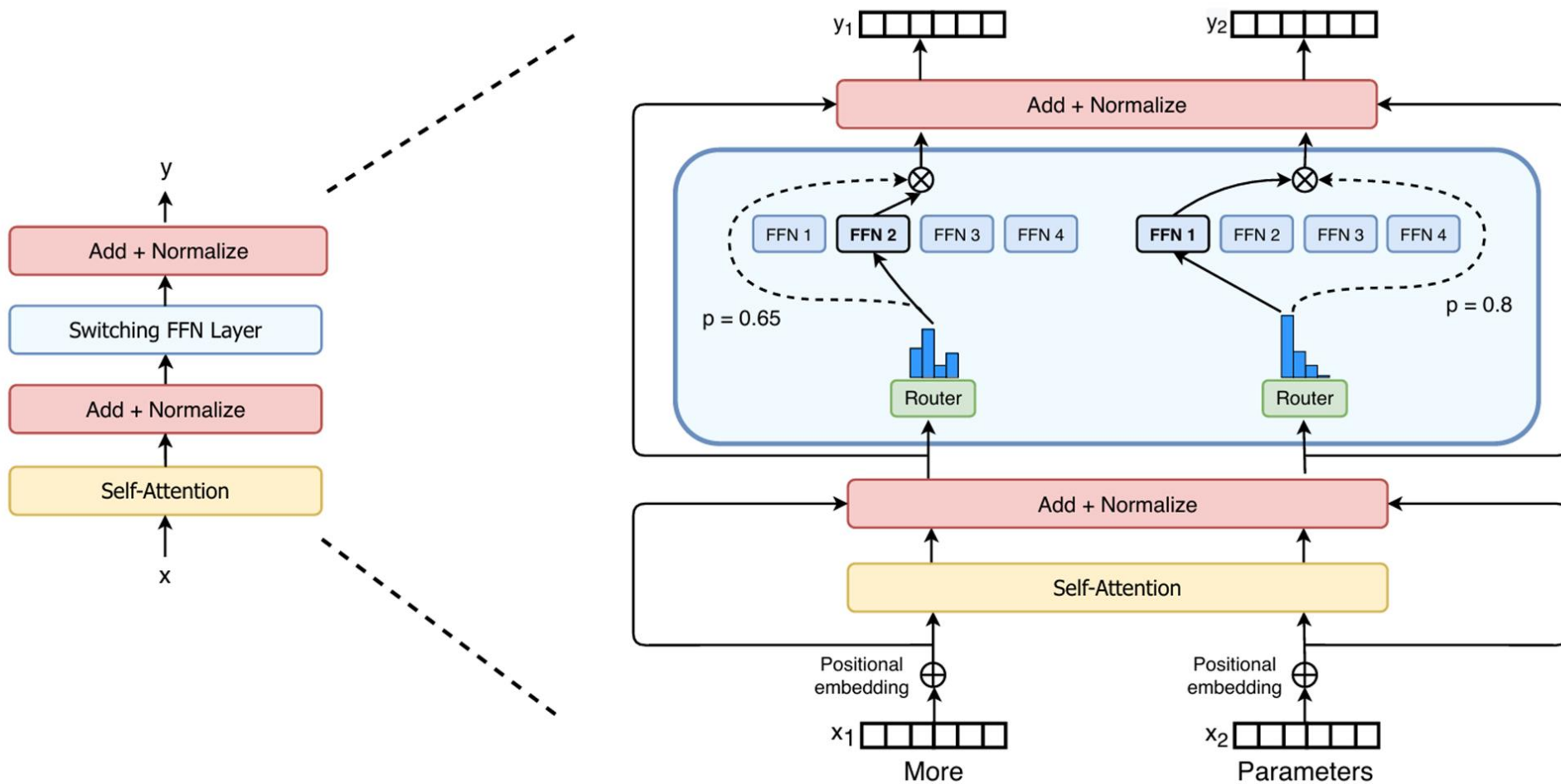
- 这样就得到了第 l 层的输出 $\mathbf{h}_t^l$ ，作为下一层的输入
- 完整流程：

$$\mathbf{h}_t^l = \sum_{i=1}^N \left(g_{i,t} \text{FFN}_i(\mathbf{u}_t^l) \right) + \mathbf{u}_t^l,$$

$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N\}, K), \\ 0, & \text{otherwise,} \end{cases}$$

$$s_{i,t} = \text{Softmax}_i \left(\mathbf{u}_t^{lT} \mathbf{e}_i^l \right),$$

Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity



DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models

- 细粒度专家

刚才的SwitchTransformers里的专家大小与dense model里一样大

DeepSeekMoE对专家进行细粒度切分

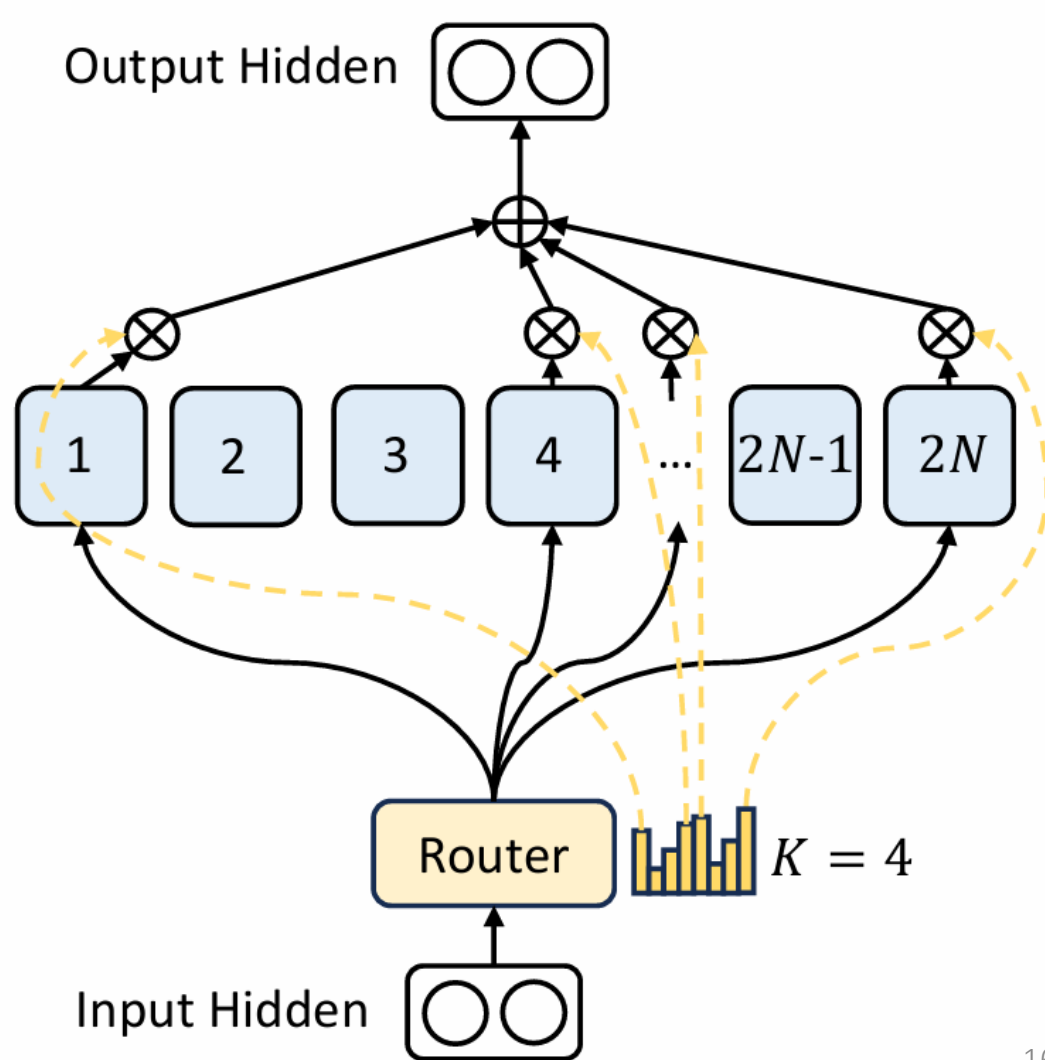
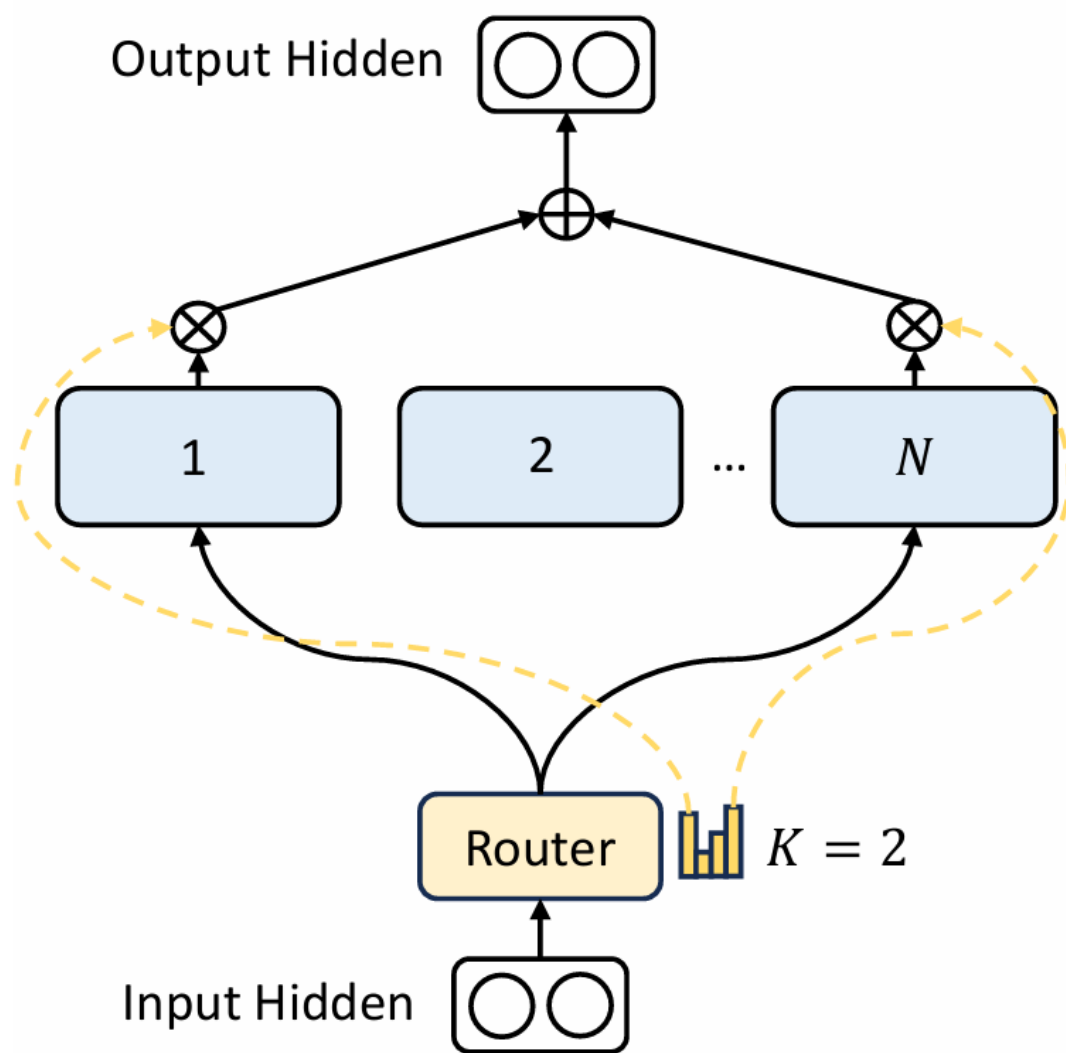
- 在专家数量有限时

分配给某个专家的token可能覆盖了太多知识

- 当我们对专家做细粒度切分

每个专家就可以学到更专业、更集中、更细粒度的知识

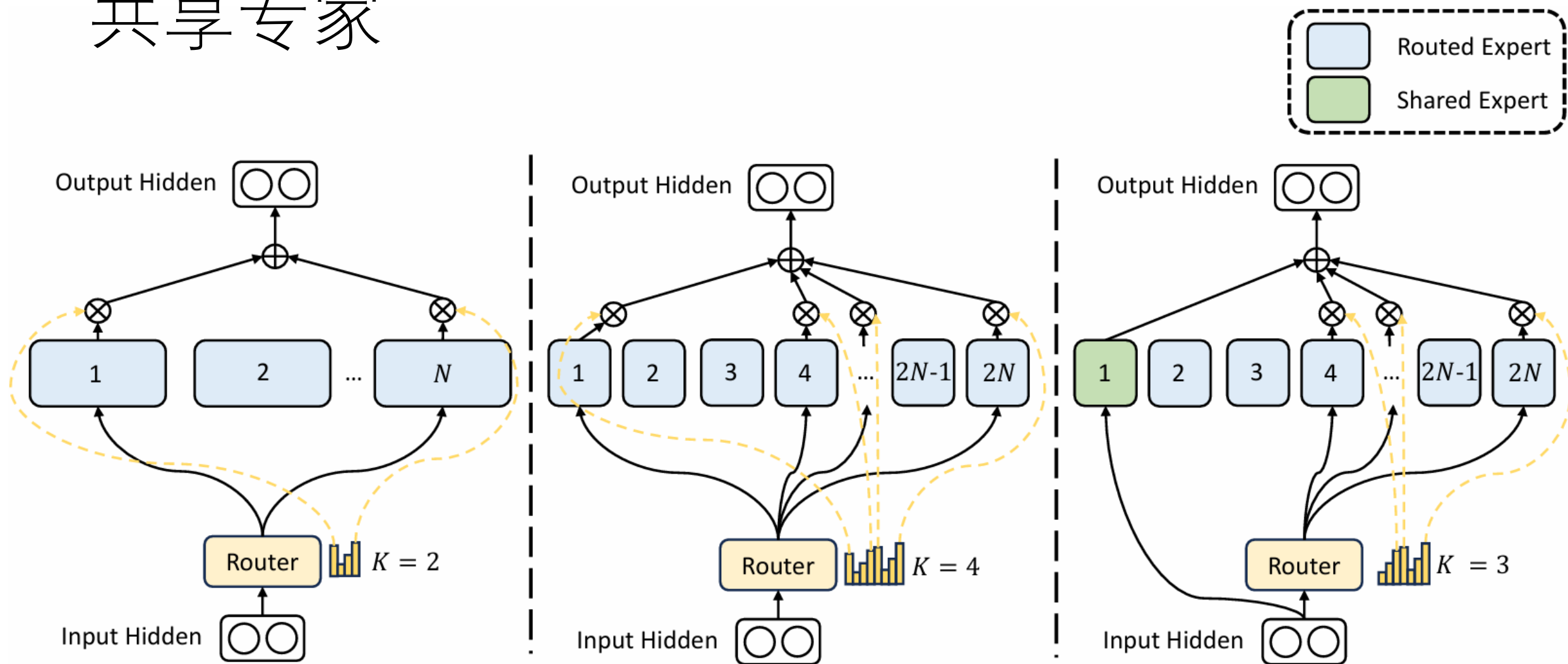
细粒度专家



Shared Expert

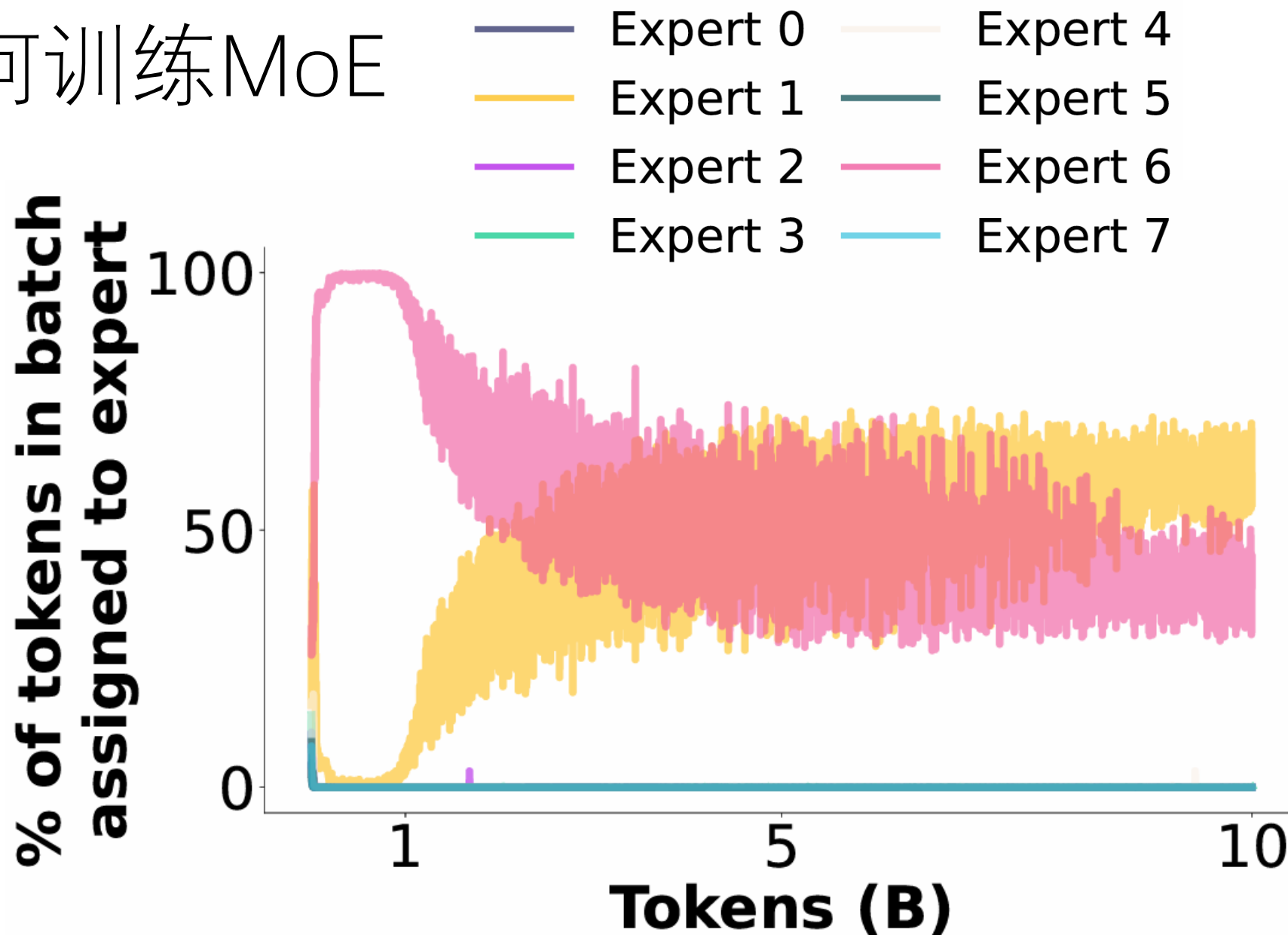
- 共享专家
- 在MoE过程中
 - 分配给不同专家的token很可能需要共同的信息
 - 每个专家为了处理自己分到的token，被迫学习通用知识
- 如何解决？
 - 选出一些专家专门处理这些通用知识
 - 这样，其他专家就可以专心学习专业知识
- 共享专家不需要router分配
 - 他是默认激活的

共享专家



(a) Conventional Top-2 Routing ➡ (b) + Fine-grained Expert Segmentation ➡ (c) + Shared Expert Isolation (DeepSeekMoE)

如何训练MoE



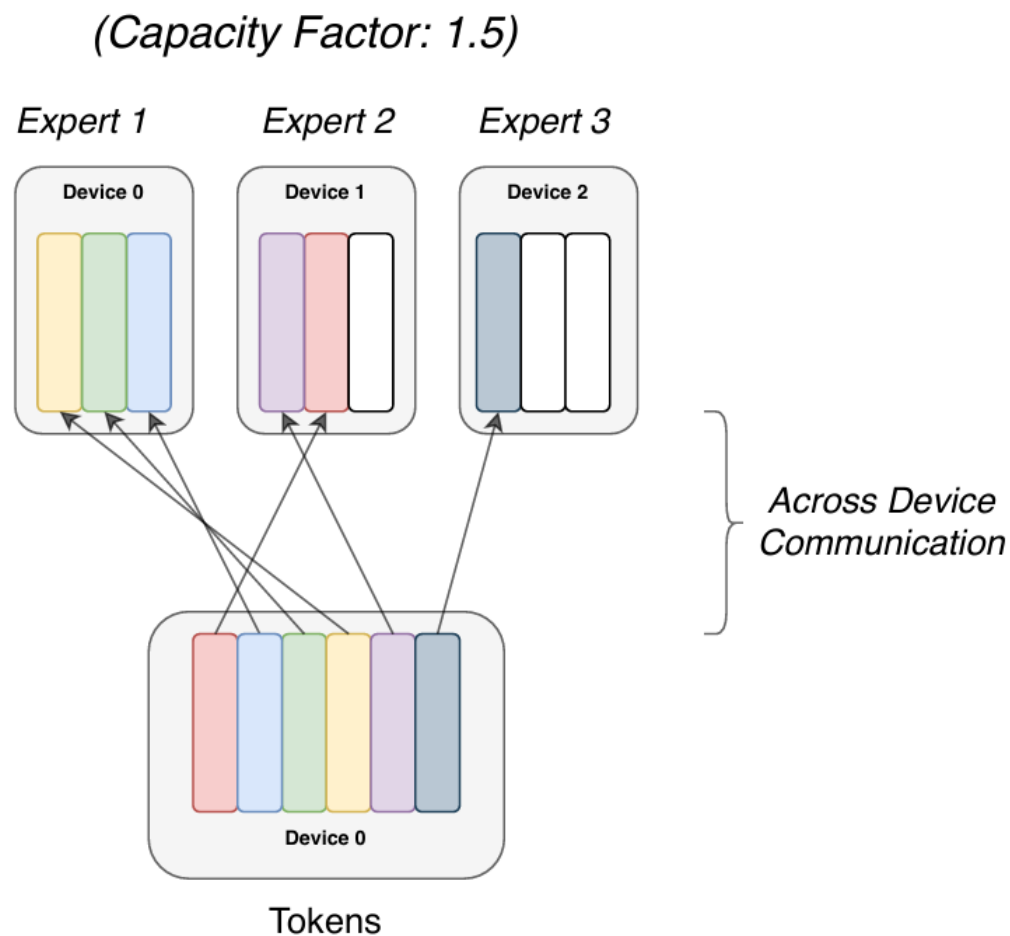
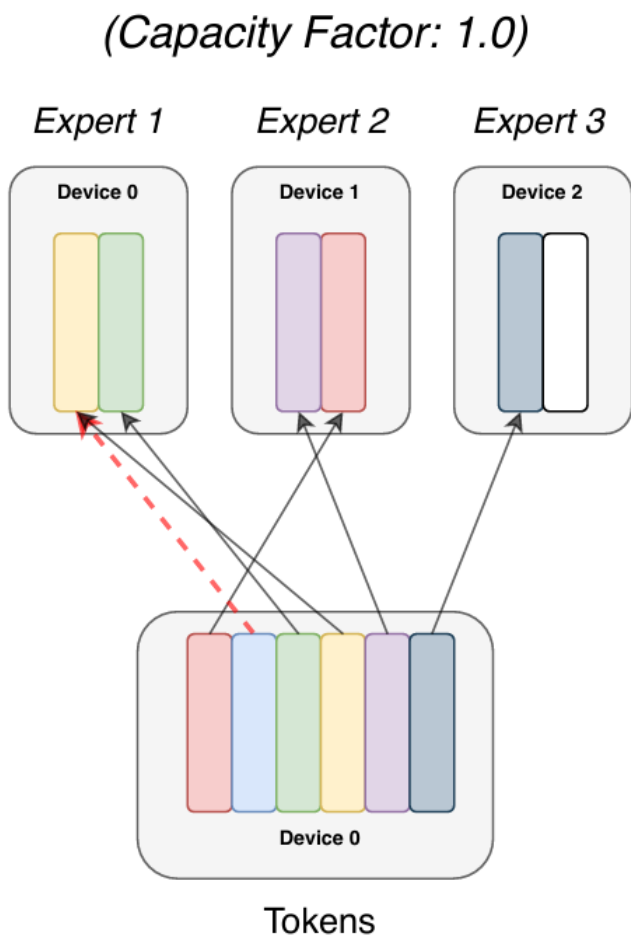
Auxiliary Load Balancing Loss

- 负载均衡损失
- 我们想让每个专家的负载尽量均衡
- $loss = \alpha \cdot N \cdot \sum_{i=1}^N f_i \cdot P_i$
 - α 是超参数, N 是专家的数量
 - f_i 是第*i*个专家的实际负载比例, P_i 是路由概率期望
- 两个分布的点积在均匀分布时最小
 - 最小化这个loss, 强迫 f_i 和 P_i 接近均匀

Expert Capacity

- 限制每个专家的最大吞吐量

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}$$



强制要求每个专家尽量均衡

- Switch Transformers 强制要求每个专家一样忙
比如某件困难的任务只能这个专家做
但是由于负载均衡损失的存在，强行分配给另一个专家
- **DeepSeekMoE**的改进

$$\mathcal{L}_{\text{ExpBal}} = \alpha_1 \sum_{i=1}^{N'} f_i P_i,$$

$$f_i = \frac{N'}{K'T} \sum_{t=1}^T \mathbb{1}(\text{Token } t \text{ selects Expert } i),$$

$$P_i = \frac{1}{T} \sum_{t=1}^T s_{i,t},$$

$$\mathcal{L}_{\text{DevBal}} = \alpha_2 \sum_{i=1}^D f'_i P'_i,$$

$$f'_i = \frac{1}{|\mathcal{E}_i|} \sum_{j \in \mathcal{E}_i} f_j,$$

$$P'_i = \sum_{j \in \mathcal{E}_i} P_j,$$

DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model

- Device-Limited Routing

限制一个token最多只能去 M 个device

Top- M

- 完整流程

For DeepSeek-V2, beyond the naive top- K selection of routed experts, we additionally ensure that the target experts of each token will be distributed on at most M devices. To be specific, for each token, we first select M devices that have experts with the highest affinity scores in them. Then, we perform top- K selection among experts on these M devices. In practice, we find that when $M \geq 3$, the device-limited routing can achieve a good performance roughly aligned with the unrestricted top- K routing.

DeepSeek-V3 Technical Report

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

- Auxiliary-Loss-Free Load Balancing

彻底砍掉用来让专家们均衡的损失函数
在线学习 调节权重

speed. Through the dynamic adjustment, DeepSeek-V3 keeps balanced expert load during training, and achieves better performance than models that encourage load balance through pure auxiliary losses.

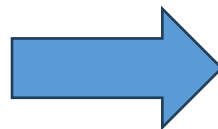
V3: 砍掉了， 但没完全砍掉

Complementary Sequence-Wise Auxiliary Loss. Although DeepSeek-V3 mainly relies on the auxiliary-loss-free strategy for load balance, to prevent extreme imbalance within any single sequence, we also employ a complementary sequence-wise balance loss:

- 由于 b_i 的调整是在线且基于统计的
即使大部分情况下可以很好调节负载至均衡
其反应也一定具有滞后性
- 训练的时候砍掉Auxiliary Loss没问题
但是推理的时候
如果专家收到了严重偏离分布的序列，专家会不知所措
- 推理阶段需要更强大的平衡机制
所以引入Complementary Sequence-Wise Auxiliary Loss

softmax after TopK

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t),$$
$$g_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise,} \end{cases}$$
$$s_{i,t} = \text{Softmax}_i(\mathbf{u}_t^T \mathbf{e}_i),$$



$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \text{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \text{FFN}_i^{(r)}(\mathbf{u}_t),$$
$$g_{i,t} = \frac{g'_{i,t}}{\sum_{j=1}^{N_r} g'_{j,t}},$$
$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} \in \text{Topk}(\{s_{j,t} | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise,} \end{cases}$$
$$s_{i,t} = \text{Sigmoid}(\mathbf{u}_t^T \mathbf{e}_i),$$